CSA1221- COMPUTER ARCHITECTURE

NAME : SARU HASINI.S

REG NO : 192521418

Application Based

1. Smart City Surveillance System

Answer:

A smart city surveillance system continuously transfers video data between **I/O devices (cameras), memory (RAM), and the CPU**. During peak hours, multiple cameras generate large amounts of data simultaneously, causing congestion.

Interaction between CPU, Memory, and I/O

- **I/O Devices:** Cameras capture and send video streams.
- **Memory (RAM):** Temporarily stores incoming video frames.
- **CPU:** Fetches data from memory, processes it, and performs threat detection.

If the CPU cannot process data as fast as it arrives, memory becomes overloaded, leading to delays.

Single-Bus vs Multi-Bus Structure

Single-Bus System

- CPU, memory, and I/O share one common bus.
- Only one data transfer occurs at a time.
- Causes bus contention and slower performance.

Multi-Bus System

- Separate buses are used for CPU-memory and I/O communication.
- Multiple transfers occur simultaneously.
- Reduces waiting time and improves throughput.

Improving the Instruction Execution Cycle

- Use **instruction pipelining** to overlap fetch, decode, and execute stages.
- Increase **cache memory** to reduce RAM access time.
- Use **DMA (Direct Memory Access)** for transferring video data directly to memory.
- Employ **multi-core processors** to process multiple video streams simultaneously.

Conclusion

A multi-bus architecture, larger cache, DMA, and pipelining significantly reduce processing delays and improve real-time surveillance performance.

2. Real-Time Stock Trading Application

Answer:

The stock trading application executes millions of instructions every second. High CPI (Cycles Per Instruction) increases transaction delays.

Impact of the Fetch–Decode–Execute Cycle

1. **Fetch**

- CPU retrieves instruction from memory.
- Cache misses increase fetch time.

2. **Decode**

- CPU interprets the instruction.
- Complex instructions increase decoding time.

3. **Execute**

- Arithmetic, memory access, or branch operations are performed.
- Branch instructions may flush the pipeline, increasing CPI.

Reasons for High CPI

- Frequent memory access
- Cache misses
- Branch instructions
- Pipeline stalls

Methods to Reduce CPI

- Increase cache size.
- Use branch prediction.
- Apply instruction pipelining.
- Use superscalar processors.
- Optimize software to reduce unnecessary memory access.
- Improve compiler optimization.

Conclusion

Reducing memory delays and branch penalties lowers CPI, increases CPU performance, and enables faster transaction processing.

3. 8085 Microprocessor – Industrial Temperature Control

Answer:

The 8085 microprocessor collects sensor readings and controls actuators. Incorrect addressing modes may result in wrong temperature values being processed.

Addressing Modes and Their Influence

Immediate Addressing

- Data is directly included in the instruction.
- Example:

MVI A,25H

Register Addressing

- Data is stored in CPU registers.
- Faster execution.

Direct Addressing

- Memory address is explicitly specified.
- Example:

LDA 2500H

Indirect Addressing

- Memory address is stored in a register pair.
- Example:

MOV A,M

Possible Causes of Errors

- Incorrect memory address
- Wrong register selection
- Uninitialized registers
- Sensor data stored in incorrect locations
- Programming mistakes

Using the 8085 Instruction Set Effectively

- Use correct addressing modes.

- Validate sensor readings.
- Initialize registers before use.
- Use comparison instructions (CMP).
- Handle abnormal readings using conditional jumps.
- Test the program thoroughly.

Conclusion

Proper addressing modes and correct instruction usage ensure accurate temperature monitoring and reliable industrial control.

4. 8086 Segmentation

Answer:

The 8086 microprocessor uses segmented memory to access up to 1 MB of memory.

Memory Segments

Code Segment (CS)

- Stores program instructions.

Data Segment (DS)

- Stores variables and data.

Stack Segment (SS)

- Stores function calls, return addresses, and local variables.

Extra Segment (ES)

- Used for additional data storage.

Problems Due to Improper Segment Handling

- Wrong data accessed.
- Stack overflow.
- Incorrect return addresses.
- Program crashes.
- Memory corruption.

Instruction Execution

The CPU calculates the physical address using:

Physical Address = Segment × 16 + Offset

Example:

DS = 2000H

Offset = 1200H

Physical Address = 21200H

Addressing Modes

- Immediate
- Register
- Direct
- Based
- Indexed
- Based Indexed

Correct use prevents memory access errors.

Ensuring Correct Execution

- Initialize segment registers correctly.
- Prevent stack overflow.
- Use PUSH and POP properly.
- Validate memory addresses.
- Use proper addressing modes.

Conclusion

Correct segment management ensures reliable execution and prevents memory-related errors.

5. Number System Conversion in Data Communication

Answer:

Packet data is commonly represented in hexadecimal because it is easier to read than binary.

Importance of Hexadecimal and Binary Conversion

- Simplifies debugging.

- Makes packet analysis easier.
- Reduces representation length.
- Helps programmers identify instruction codes quickly.

Example

Hexadecimal:

3A

Binary:

00111010

Effects of Conversion Errors

- Wrong instruction execution.
- Incorrect packet interpretation.
- Data corruption.
- Communication failures.
- Security vulnerabilities.

Role of Efficient CPU Design

- Faster instruction execution.
- Improved cache performance.
- Better error handling.
- Higher throughput.

Role of Benchmarking

Benchmarking measures:

- CPU execution time
- Memory performance
- Cache hit ratio
- Network throughput
- System latency

It helps identify bottlenecks and optimize system performance.

Conclusion

Accurate hexadecimal-to-binary conversion, efficient CPU architecture, and regular benchmarking improve system reliability and minimize processing errors.